

DigitalPulse

統合型心電図解析・患者管理システム

ER図（データベース設計書）

文書番号	DP-ERD-001
版数	1.0
作成日	2026年5月5日
作成者	辰田
上位文書	DP-DD-001 詳細設計書

本書は学術・ポートフォリオ目的で作成されています

改訂履歴

版数	日付	作成／改訂者	改訂内容
1.0	2026年5月5日	辰田	初版作成

目次

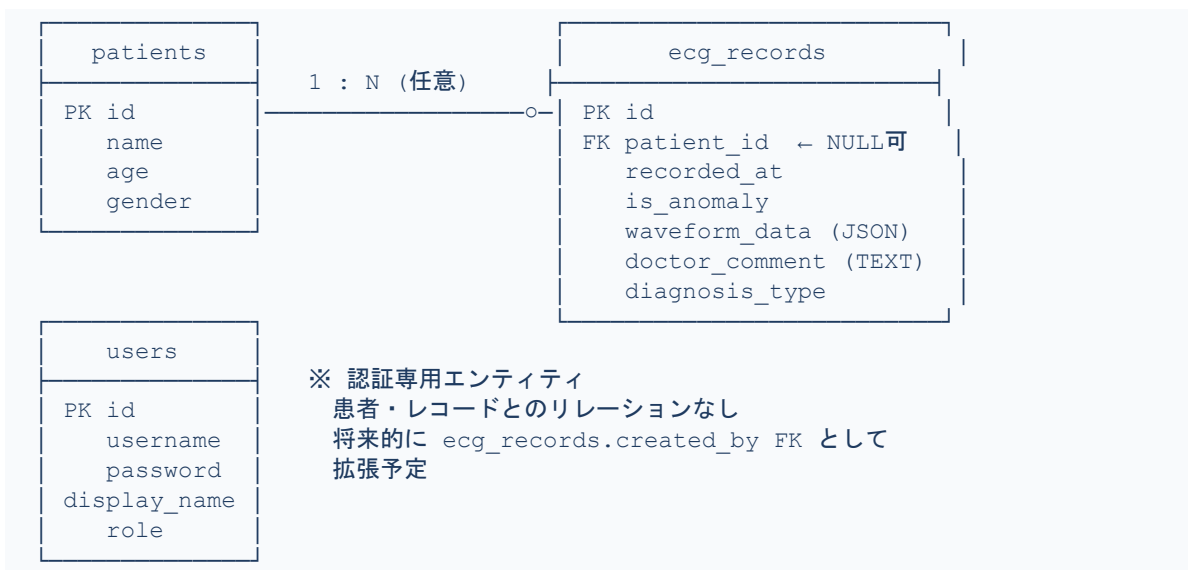
改訂履歴	2
目次	3
1. データベース概要	4
2. リレーション図（ER図）	4
3. テーブル定義	5
3.1 patients テーブル	5
3.2 ecg_records テーブル	5
3.3 users テーブル	6
4. DDL（テーブル作成 SQL）	7
4.1 patients	7
4.2 ecg_records	7
4.3 users	7
5. インデックス設計	8
6. 設計上の判断・ポイント	9
6.1 patient_id を NULL 許容にした理由	9
6.2 waveform_data を JSON 型で保存した理由	9
6.3 users テーブルとのリレーションを持たない理由	9
6.4 ON DELETE SET NULL の選択	9
7. 承認欄	10

1. データベース概要

DigitalPulse のデータベースは MySQL 8.x (Railway マネージドサービス) 上で運用する。テーブルは3つで構成され、患者情報・心電図レコード・ユーザー認証情報をそれぞれ独立したテーブルで管理する。Spring Data JPA (Hibernate) を介してアクセスし、スキーマ管理は `ddl-auto=none` で手動管理する。

2. リレーション図（ER図）

以下に、テーブル間のリレーションを示す。



記号・凡例	説明
———○—	1 対 多（任意）：1人の患者が 0件以上の心電図レコードを持つ
🔑 PK	Primary Key（主キー）：AUTO_INCREMENT で自動採番
🔗 FK	Foreign Key（外部キー）：patients.id を参照。NULL 許容

3. テーブル定義

3.1 patients テーブル

患者の基本情報を管理するマスターテーブル。ecg_records から参照される親テーブルであり、患者 1 人に対して複数の心電図レコードが紐づく（1対多）。

patients		
カラム名	型	制約・説明
 PK id	INT	PRIMARY KEY / AUTO_INCREMENT / NOT NULL
name	VARCHAR(255)	NOT NULL 患者氏名
age	INT	NULL 許容 年齢
gender	VARCHAR(50)	NULL 許容 Male / Female / Other

Java エンティティクラス : *Patient.java* (@Entity @Table(name="patients"))

3.2 ecg_records テーブル

心電図の 1 記録を管理するメインテーブル。AI 推論結果・波形データ・医師コメントを統合して保持する。patient_id は NULL 許容とし、患者未指定でのインポートを許容する設計とした。

ecg_records		
カラム名	型	制約・説明
 PK id	INT	PRIMARY KEY / AUTO_INCREMENT / NOT NULL
 FK patient_id	INT	FOREIGN KEY → patients.id / NULL 許容（患者未指定インポート対応）
recorded_at	DATETIME	NULL 許容 インポート時刻 (LocalDateTime)
heart_rate	INT	NULL 許容 心拍数（将来拡張用）
is_anomaly	TINYINT(1)	NULL 許容 AI判定による異常フラグ (true=1 / false=0)
waveform_data	JSON	NOT NULL 187点の波形データ（JSON配列文字列）
doctor_comment	TEXT	NULL 許容 AI所見テキスト+医師による修正・追記内容
diagnosis_type	INT	NULL 許容 診断分類コード（0:Normal / 1:SVEB / 2:VEB / 3:Fusion / 4:Unknown）

Java エンティティクラス : *EcgRecord.java* (@Entity @Table(name="ecg_records"))

diagnosis_type の値はAIサービスの CLASS_NAMES ディクショナリ（0～4）と対応する。

3.3 users テーブル

システムログイン用のユーザー認証情報を管理するテーブル。現バージョンでは patients・ecg_records テーブルとの直接リレーションは持たない独立エンティティである。将来的に「誰が所見を記録したか」のトレーサビリティが必要になった際は、ecg_records に created_by カラム (FK → users.id) を追加することで拡張できる。

users		
カラム名	型	制約・説明
 PK id	INT	PRIMARY KEY / AUTO_INCREMENT / NOT NULL
username	VARCHAR(255)	UNIQUE / NOT NULL ログインユーザー名（重複不可）
password	VARCHAR(255)	NOT NULL パスワード（将来的に bcrypt ハッシュ化を予定）
display_name	VARCHAR(255)	NULL 許容 画面表示名 (@Column(name="display_name"))
role	VARCHAR(50)	NULL 許容 ユーザー権限（将来の認可制御に使用予定）

Java エンティティクラス : User.java (@Entity @Table(name="users"))

4. DDL（テーブル作成 SQL）

以下に各テーブルの CREATE TABLE 文を示す。ddl-auto=none により Hibernate の自動スキーマ変更は禁止しており、このDDLを手動実行してスキーマを管理する。

4.1 patients

```
CREATE TABLE patients (  
  id      INT          NOT NULL AUTO_INCREMENT,  
  name    VARCHAR(255) NOT NULL,  
  age     INT,  
  gender  VARCHAR(50),  
  PRIMARY KEY (id)  
);
```

4.2 ecg_records

```
CREATE TABLE ecg_records (  
  id              INT          NOT NULL AUTO_INCREMENT,  
  patient_id      INT,  
  recorded_at     DATETIME,  
  heart_rate      INT,  
  is_anomaly      TINYINT(1),  
  waveform_data   JSON          NOT NULL,  
  doctor_comment  TEXT,  
  diagnosis_type  INT,  
  PRIMARY KEY (id),  
  FOREIGN KEY (patient_id)  
    REFERENCES patients(id)  
    ON DELETE SET NULL  
    ON UPDATE CASCADE  
);
```

4.3 users

```
CREATE TABLE users (  
  id          INT          NOT NULL AUTO_INCREMENT,  
  username     VARCHAR(255) NOT NULL,  
  password     VARCHAR(255) NOT NULL,  
  display_name VARCHAR(255),  
  role         VARCHAR(50),  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_username (username)  
);
```

5. インデックス設計

検索パフォーマンスを最適化するため、以下のインデックスを設定する。

テーブル	対象カラム	理由
users	username (UNIQUE)	ログイン時のユーザー名検索 (findByUsername) に使用
ecg_records	patient_id (FK)	患者別レコード検索 (searchRecords / patientId フィルタ) の高速化
ecg_records	is_anomaly	異常フラグによる絞り込み検索 (searchRecords / anomaly フィルタ) の高速化
ecg_records	id (降順)	findAllByIdDesc() でのダッシュボード全件取得の高速化

※補足 ページネーション (OFFSET/LIMIT) を高速に動作させるため、id DESC のインデックスが必須である。

6. 設計上の判断・ポイント

本章は就活面接での「DB設計で工夫した点・意識した点」の回答を文書化したものである。

6.1 patient_id を NULL 許容にした理由

心電図データのインポート時に、患者情報の登録が完了していないケースを想定した。NULL 許容にすることで「まずデータを取り込み、後から患者紐づけを行う」という現場の運用フローに対応できる。完全な正規化（NOT NULL 強制）よりも、ユースケースに合わせた柔軟性を優先した設計判断である。

6.2 waveform_data を JSON 型で保存した理由

心電図の 187 点データは固定長の数値配列である。別テーブルに正規化する方法（ecg_points テーブルに 187 行挿入）も検討したが、以下の理由で JSON 型カラムへの一括格納を選択した。

- ▶ 1 レコードあたり 187 行の INSERT が不要になり、インポート処理が大幅に高速化される
- ▶ 波形データは参照のみで、個別ポイントへのクエリが不要なため、JSON 型で十分なアクセスパターンに合致する
- ▶ MySQL 8.x の JSON 型は JSON_EXTRACT 等による部分参照も可能であり、将来の拡張にも対応できる

6.3 users テーブルとのリレーションを持たない理由

現バージョンでは「誰がログインしているか」の認証のみが要件であり、「誰が所見を記録したか」の監査ログは要件外である。不要なリレーションを追加することでJOINのコストが増加することを避け、シンプルな構成を維持した。将来的に監査要件が加わった場合は、ecg_records に created_by FK を追加するだけで対応可能な設計になっている。

6.4 ON DELETE SET NULL の選択

患者情報が削除された場合に、紐づく心電図レコードも連鎖削除（ON DELETE CASCADE）するか、患者IDをNULL化（ON DELETE SET NULL）するかを検討した。医療データの性質上、患者情報が削除されても心電図レコード自体は保全すべきと判断し、ON DELETE SET NULL を採用した。

7. 承認欄

役割	氏名	署名	承認日
作成者	辰田		2026年5月5日
レビュー者			
承認者			

以上